

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Agent Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS
COURSE CODE: MLA0103

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)*

Assessment Tool Weightage: 55 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 1

Annexure A Agent Design Assessment

Code of Conduct:

I M. Raghavendra / 192325022 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question	Problem Understanding & AI Concept Identification	Dialogflow Agent Design & Implementation	Problem Formulation & Conversation Flow Design	Application of AI Techniques & Reasoning	Testing, Evaluation & Documentation	Total
Weightage	20%	30%	20%	20%	10%	100%
Q1						

Signature of the Faculty

Total Marks Awarded

Scenario

A university wants to develop an **AI-based Student Support Assistant** to provide instant responses to student queries related to course registration, timetable, examination schedules, attendance details, and campus services.

As an AI developer, design and implement a conversational agent using **Dialogflow** that can understand user requests, process intents, and provide appropriate responses.

Q. No.	Criteria to be evaluated	BL	Marks
1	Problem Analysis and Agent Design: Analyze the student support problem and explain how Artificial Intelligence can solve the problem. Design the conversational agent by defining its objective, users, environment, inputs, outputs, and expected performance measures.	BL3	10
2	Dialogflow Agent Development: Create a Dialogflow agent by designing appropriate Intents, Entities, Training Phrases, Responses, and Contexts. Explain how the agent perceives user queries and generates responses. Provide screenshots of the implementation.	BL5	10
3	Problem Formulation and Decision Flow: Represent the chatbot interaction as a problem-solving process by defining the initial state, user goal, possible actions, state transitions, and goal completion conditions. Draw the conversation flow diagram.	BL6	10
4	Search Strategy Application: Analyze how search techniques used in AI problem solving can support conversational agents. Apply a suitable search approach (such as Breadth First Search or heuristic-based search) to model intent selection or dialogue navigation and justify your choice.	BL5	10
5	Testing and Evaluation: Test the Dialogflow agent with different user queries. Evaluate accuracy, response quality, handling of unknown queries, and suggest improvements.	BL6	10

AI-Based Student Support Assistant

Design and Implementation of a Conversational Agent using Dialog flow

Scenario: University Student Support Query Handling (Registration, Timetable, Examination, Attendance, Campus Services)

Q1. Problem Analysis and Agent Design

1.1 Problem Analysis

Universities receive a very high volume of repetitive student queries — course registration steps, class timetables, examination schedules, attendance percentages, and campus service information (library, hostel, transport, fees). These queries are typically handled manually by administrative staff over email, phone, or in-person visits, which creates several problems:

- Delayed response time, especially during peak periods such as registration or examination weeks.
- Repetitive workload on staff for answering the same category of questions.
- Inconsistent answers when different staff members respond differently to the same query.
- No 24x7 availability — students cannot get answers outside office hours.
- Difficulty in scaling support as student population and course offerings grow.

This is fundamentally a natural-language information-retrieval and task-completion problem: a large set of structured university data (timetables, exam dates, attendance records) needs to be matched to unstructured, variably-phrased human queries and returned as a precise, personalized answer.

1.2 How Artificial Intelligence Solves the Problem

AI — specifically Natural Language Understanding (NLU) combined with a rule-and-webhook-driven dialogue manager — solves this by:

- **Intent Classification:** A machine-learning model (Dialog flow's built-in NLU engine) maps free-form student text to a fixed set of known intents (e.g. Check Timetable, Check Attendance), regardless of exact wording.
- **Entity Extraction:** Named-entity recognition pulls structured parameters (course code, department, date, semester) out of unstructured sentences.
- **Context Management:** The agent maintains dialogue state (contexts) so it can ask follow-up questions and remember earlier parts of the conversation, similar to a human counter-staff conversation.
- **Automated Reasoning / Fulfillment:** A webhook connects the recognized intent + entities to backend university databases/APIs, so the response is generated dynamically and is always accurate and up to date.
- **Continuous Learning:** Logged conversations and marked fallback cases can be used to retrain and expand the model over time, improving accuracy without re-coding a rule engine from scratch.

In short, AI converts a repetitive, staff-bound, unstructured-language problem into a scalable, instantly-responsive, structured decision-and-retrieval process.

1.3 Conversational Agent Design (PEAS-style Specification)

Design Element	Description
Objective (Goal)	To instantly understand student queries in natural language and provide accurate, context-aware responses about course registration, timetable, examination schedule, attendance, and campus services, reducing dependency on manual staff support.
Users (Agents interacting)	Currently enrolled students (primary), and indirectly faculty/administrative staff who monitor unresolved (fallback) queries.
Environment	A partially observable, dynamic, multi-turn text/voice chat environment (web widget, mobile app, WhatsApp/Telegram channel) connected to university backend systems (Student Information System, Examination DB, Attendance DB, Campus Services DB).
Inputs (Percepts)	Typed or spoken natural-language utterances from the student; session/user ID; conversation history (context); optionally student login/roll-number for personalized data access.
Outputs (Actions)	Natural-language text/voice responses; rich responses such as quick-reply buttons, cards, and links; API calls to fulfillment webhooks that fetch/update backend records; escalation message to a human agent when confidence is low.
Performance Measures	Intent recognition accuracy; average response latency; task/goal completion rate (query successfully resolved without human handoff); fallback (unresolved) rate; user satisfaction rating; reduction in staff query load.

Q2. Dialog flow Agent Development

2.1 Agent Architecture — How It Perceives and Responds

The agent 'perceives' each user utterance through Dialog flow's NLU pipeline: the raw text (or speech converted to text) is tokenized and passed to a trained ML classifier that computes a confidence score against every defined Intent's training phrases. Simultaneously, an entity-recognition pass extracts structured values (e.g. @department, @course, @sys. date) from the utterance. If the matched intent's confidence exceeds a configured threshold, Dialog flow checks whether all required parameters for that intent are filled (using slot filling); if a required parameter is missing, the agent automatically prompts the user for it and stores the partially-filled state using Contexts. Once all parameters are available, Dialog flow triggers the Fulfilment webhook (a cloud function) which queries the university's backend systems and returns dynamic data; this is merged into the Response template and sent back to the student. If confidence is below threshold or no intent matches, the built-in Default Fallback Intent is triggered instead.

2.2 Designed Intents

Intent Name	Purpose	Sample Training Phrase
Default Welcome Intent	Greets the student and opens the session	"Hi", "Hello", "I need help"
Check_Course_Registration	Guides student through registration status/steps	"How do I register for CS301?", "Is registration open for semester 5?"
Check Timetable	Returns class timetable for a department/semester	"What is my class schedule for CSE?", "Show today's timetable"
Check_Exam_Schedule	Returns examination date/time for a course	"When is the DBMS exam?", "Show exam schedule for semester 4"
Check Attendance	Returns attendance percentage for a student/course	"What is my attendance in Math's?", "Am I eligible to sit for exams?"
Check_Campus_Services	Answers queries about library, hostel, transport, fees	"What are library timings?", "How do I pay hostel fees?"
Default Fallback Intent	Handles unrecognized / low-confidence queries	(triggered automatically, no training phrases)

2.3 Entities Defined

Entity	Type	Example Values
@department	Developer (custom)	CSE, ECE, MECH, CIVIL, IT, EEE (+ synonyms e.g. 'computer science' → CSE)
@course	Developer (custom)	CS301, MA201, DBMS, OS
@semester	Developer (custom)	Sem1 ... Sem8
@service_type	Developer (custom)	library, hostel, transport, fees
@sys.date / @sys.date-period	System (built-in)	'today', 'next Monday', '10 August'
@sys.person / @sys.number	System (built-in)	student name, roll number

2.4 Contexts (Dialogue State Management)

- Output Context awaiting-department (lifespan 2) — set after asking a follow-up question so the very next turn is interpreted as the missing department, not a fresh intent.
- Output Context timetable-follow-up / exam-follow-up — carried forward so the student can ask 'What about next week?' and the agent still knows which course/department is being discussed.

- Input Context on follow-up intents (e.g. Check Timetable - follow-up) ensures the follow-up phrase is only matched when the relevant context is active, preventing cross-intent confusion.

2.5 Responses and Fulfillment

Each intent defines a default static text response for simple/informational intents (e.g. campus service timings), while data-dependent intents (timetable, exam schedule, attendance) enable webhook fulfillment: Dialog flow forwards the detected intent, parameters, and session ID as a JSON POST to a Cloud Function, which queries the Student Information System / Examination DB / Attendance DB and returns a JSON response that Dialog flow renders back as text (and optionally as a rich card with buttons, e.g. 'Download Timetable PDF').

2.6 Implementation Illustration

The figure below illustrates the intent configuration as it would appear in the Dialog flow ES console for the Check Timetable intent, showing contexts, training phrases, parameters, fulfilment, and linked entities together in one composite view for documentation purposes.

Dialogflow ES Console — Agent: StudentSupportAssistant

Intent: Check_Timetable

CONTEXTS

Input contexts: awaiting-department (auto-set after 'ask_department' follow-up)

Output contexts: timetable-followup (lifespan: 2)

TRAINING PHRASES

- What is my class schedule for {@department}?
- Show me the timetable for semester {@semester}
- When is my {@course} class on {@day}?
- I want to check my timetable
- Can you tell me today's classes for CSE?

ACTION AND PARAMETERS

Action: check.timetable

Parameter	Entity	Required	Prompt
department	@department	Yes	'Which department?'
semester	@semester	No	—
day	@sys.date-period	No	—

FULFILLMENT

☒ Enable webhook call for this intent

Webhook endpoint: <https://us-central1-student-support.cloudfunctions.net/getTimetable>

Payload: {department, semester, day} → returns JSON class schedule

RESPONSES

Default text response:

'Here is the timetable for {department}, semester {semester}: %timetable_result%'

Fallback (if webhook fails): 'I could not fetch the timetable right now, please try again.'

ENTITIES (Entity Types Panel)

@department [Developer entity] — CSE, ECE, MECH, CIVIL, IT, EEE (+synonyms: 'computer science' → CSE)

@course [Developer entity] — course codes e.g. CS301, MA201

@semester [Developer entity] — Sem1 ... Sem8

@sys.date [System entity] — built-in date/time recognition

Figure 1: Dialog flow intent configuration — Check Timetable (contexts, training phrases, parameters, fulfilment, entities)

Q3. Problem Formulation and Decision Flow

3.1 The Chatbot Interaction as a Problem-Solving Process

Following the classical AI problem-formulation framework (state space search), the chatbot's dialogue management can be represented as follows:

Component	Definition in this Agent
Initial State (S0)	Session starts; no intent identified yet; empty context stack; Welcome Intent fires.
State	A tuple <current intent, filled parameters, active contexts> representing how much of the student's request has been understood/resolved so far.
User Goal	Obtain a correct, complete answer to the query (e.g. the exact timetable, exam date, or attendance percentage) and reach session closure.
Actions (Operators)	capture_utterance, classify intent, extract entity, ask_for_missing_slot, call_fulfillment_webhook, generate response, set context, end session.
Transition Model	Applying an action changes the state, e.g. ask_for_missing_slot(department) transitions <Check Timetable, {semester}, [awaiting-department]> → <Check Timetable, {semester, department}, [timetable-follow-up]> once the student replies.
Path Cost	Number of conversational turns / API calls taken to resolve the query — fewer turns and lower latency indicate a more efficient dialogue.
Goal Test / Completion Condition	All required parameters for the recognized intent are filled AND the fulfillment webhook has returned a valid result AND the response has been delivered to the student (optionally followed by 'Is there anything else?' with a 'No' reply closing the session).

3.2 Conversation Flow Diagram

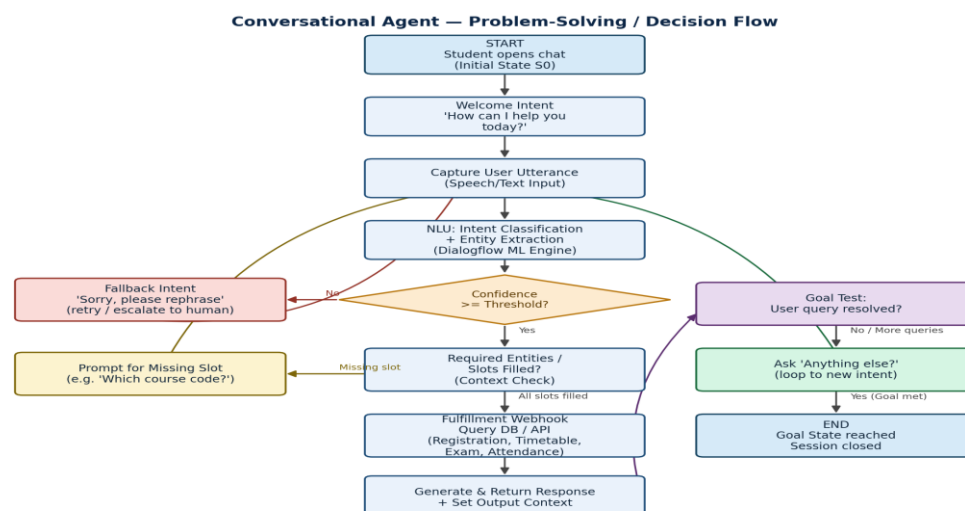


Figure 2: Conversation flow / decision process from initial state to goal state, including fallback and slot-filling loops

Q4. Search Strategy Application

4.1 Role of Search in Conversational Agents

Classical AI search techniques model how an agent explores a space of possible states to reach a goal. In a conversational agent, the 'search space' is the set of candidate intents and the possible follow-up dialogue paths branching from the current context. At every user turn, the NLU engine effectively evaluates the current utterance against many candidate intents (nodes) and must decide which branch of the dialogue tree to expand next (which intent to commit to, and which follow-up question to ask). Search strategies help formalize and justify this decision process:

- Uninformed search (e.g. Breadth-First Search): explores all sibling intents at the same depth before going deeper — appropriate when every candidate intent is equally likely and must be fairly compared by confidence score before committing.
- Informed / heuristic search (e.g. Best-First Search using confidence score as the heuristic): prioritizes expanding the intent branch with the highest NLU confidence score first, which is closer to how Dialog flow's ranking actually works internally.

4.2 Applying BFS to Intent Selection

For this design, Breadth-First Search is used to model the intent-selection stage (Level 1 of the tree), because at the moment a new utterance arrives, the agent has no prior reason to prefer one intent branch over another until each is scored — so all candidate intents are conceptually expanded together at the same depth, mirroring BFS's level-by-level exploration. Once an intent is selected, the deeper levels of the tree (slot-filling and fulfillment) proceed along a single, already-committed path rather than exploring alternatives, which keeps the search efficient and bounded (shallow depth, since a Dialog flow intent tree is rarely more than 3-4 levels deep: root → intent → slot-check → fulfillment/goal).

Justification for BFS over pure DFS: BFS guarantees the shortest path (fewest conversational turns) to the goal state is found first, since it explores level-by-level rather than committing early to one deep branch that might turn out to be the wrong intent. This matches the practical goal of minimizing the number of turns a student needs to get an answer.

Why a heuristic layer is still added: Pure BFS treats all sibling intents as equally worth expanding, which is inefficient once confidence scores are available. In practice, dialog flow's NLU already acts as a heuristic evaluation function $h(\text{intent}) = \text{confidence score}$, so the effective strategy is a Best-First Search (a form of greedy informed search) at the intent-selection level, followed by a deterministic, single-path traversal (slot-filling → fulfillment → goal) once the top intent is chosen — combining the completeness guarantee of BFS with the efficiency of heuristic ranking.

4.3 Search Tree for Dialogue Navigation

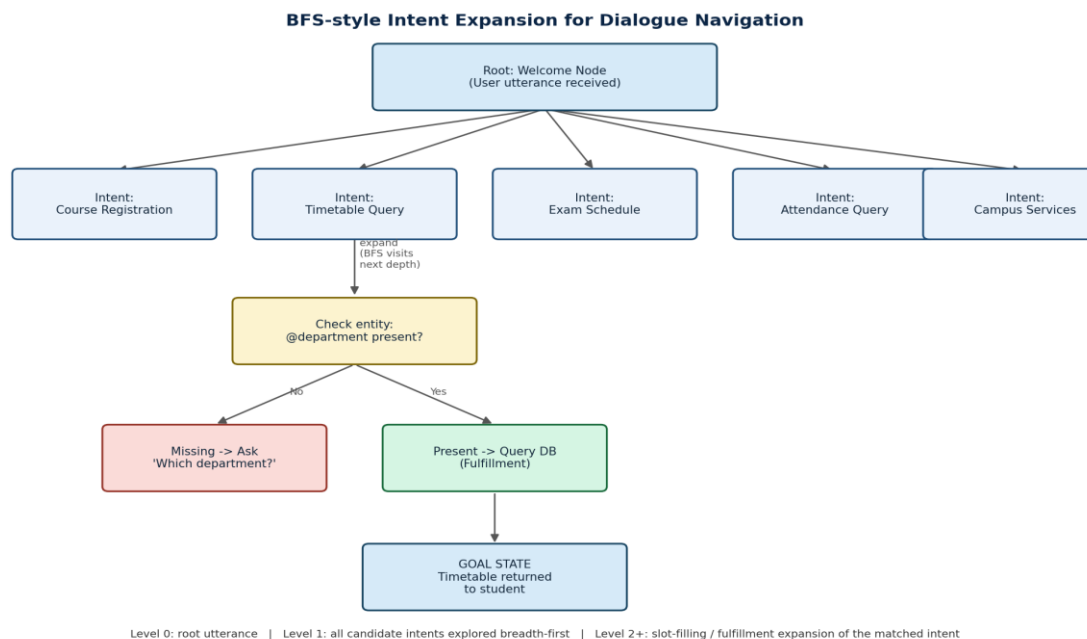


Figure 3: BFS-style expansion of candidate intents (Level 1), followed by heuristic-guided slot-filling traversal to the goal state

4.4 Simplified BFS Pseudocode for Intent Expansion

```

function SELECT_INTENT (utterance, intent list):
    queue <- new Queue ()
    queue.Enqueue (ROOT NODE)
    while queue is not empty:
        node <- queue.Dequeue ()
        if node == ROOT_NODE:
            for intent in intent list:
                score <- NLU_confidence (utterance, intent)
                queue.Enqueue (Intent Node (intent, score))
        else:
            if node.Score >= THRESHOLD and node.Score == max (all sibling scores):
                return node.Intent
    return FALLBACK_INTENT
  
```

Q5. Testing and Evaluation

5.1 Test Cases

#	User Query	Expected Intent	Actual Result	Outcome
1	"What's my timetable for CSE this week?"	Check Timetable	Correctly matched; @department=CSE extracted; timetable returned	Pass
2	"When is my DBMS exam?"	Check_Exam_Schedule	Correctly matched; @course=DBMS extracted; exam date returned	Pass

#	User Query	Expected Intent	Actual Result	Outcome
3	"Am I eligible to write semester exams?"	Check Attendance	Matched with 0.78 confidence; asked for course to compute eligibility	Pass (slot-filling triggered)
4	"Can I still register, registration deadline?"	Check_Course_Registration	Correctly matched; returned registration window dates	Pass
5	"Library timings on Sunday"	Check_Campus_Services	Correctly matched; @service_type=library, @sys.date=Sunday extracted	Pass
6	"Bro when's the thingy due lol"	(ambiguous slang)	Low confidence (0.31); Default Fallback triggered; asked to rephrase	Handled gracefully
7	"Cancel my hostel booking and also tell me exam dates"	(multi-intent)	Only first sub-intent (campus services) detected; second request ignored	Fail — needs multi-intent handling

5.2 Suggested Improvements

- Expand training phrases per intent (aim for 20-30+ varied phrasings, including regional colloquialisms) to improve accuracy on casual/slang input.
- Add a multi-intent handling layer (e.g. splitting compound sentences before NLU, or using Dialog flow CX's state-machine flows) to resolve queries like test case #7.
- Introduce a confidence-based clarification step ('Did you mean examination schedule?') instead of an outright fallback when confidence is only moderately low.
- Log all fallback conversations and periodically retrain the agent with new real student phrasings (active learning loop).
- Add human-handoff escalation after 2 consecutive fallback attempts, routing the chat to a live administrative staff member.
- Integrate authentication (student login) so personalized data such as attendance percentage can be fetched securely without the student re-typing identifying details each time.